

The field guide.

Every word you'll hear over the two days, translated into plain English. You don't need to memorise any of this. Keep it nearby, and when a term lands that you don't recognise, look it up here instead of nodding along.

Before the jargon: what an AI operating system actually is

Strip the mystique away and an AI operating system is three things. A **folder of plain text files** that describe your business: who you are, how you sound, what you sell, who you sell it to. An **AI that can read and write those files**, so every piece of work starts from your real context instead of a blank chat box. And a **handful of repeatable workflows** built on top: prospecting, proposals, content, follow-up.

That's it. Not another app. Not a subscription stack. The files are the asset. The AI is the operator. The rest of this guide explains the machinery around those two ideas.

Which model for which job

Every AI company sells roughly the same range: a big, slower, more thoughtful model, a middle everyday model, and a small fast one. The names change every few months. The tiers don't. Think of it like camera bodies: you don't bring the cinema rig to shoot a quick behind-the-scenes clip, and you don't shoot the hero brand film on your phone.

THE JOB	REACH FOR	WHY
Deep thinking. Strategy, pricing decisions, reviewing a contract, planning a quarter, untangling a messy problem.	The biggest model on offer (Claude Opus, or the top tier from OpenAI or Google).	These models reason better and miss less. Slower and dearer per use, but a pricing decision is worth thirty seconds of waiting.
Everyday writing. Emails, proposals, social posts, call summaries, meeting prep.	The middle tier (Claude Sonnet or equivalent).	Fast, capable, and cheap enough to use fifty times a day. This is where most of your week lives.
High volume, simple jobs. Tagging leads, sorting lists, pulling names out of transcripts.	The small fast tier (Claude Haiku or equivalent).	When the task is simple and you're running it hundreds of times, speed and cost win.
Research that needs sources. Checking a market, scoping a niche, background on a prospect.	Any model with web search or a deep research mode turned on.	Models answer from training data unless they can search. For anything factual and current, make it look things up.
Working inside your files. Building and running the AI OS itself.	Claude Code, with whichever model tier suits the task.	Chat tools can only see what you paste in. Claude Code reads and writes your actual folder, which is the whole point of an OS.



The rule that matters more than any model name: the quality of what comes out is set by the context that goes in. A mid-tier model with your voice files, your client list and your pricing will beat the biggest model in the world starting from a blank page.

The AI words

Model (or LLM)

The AI itself. LLM stands for large language model: a system trained on enormous amounts of text that predicts what should come next. Claude, ChatGPT and Gemini are all products built around models.

Why you care: "which model" is just "which engine". Pick by the job, not by the brand ad.

Prompt

Whatever you type to the model. A question, an instruction, a half-formed thought.

Why you care: vague prompt, vague answer. The fix is rarely a better model, it's a clearer brief, the same as briefing a freelancer.

System prompt (or custom instructions)

Standing instructions the model reads before every conversation. Your voice rules, your spelling preferences, your banned words. In an AI OS this lives in a file, which means you write it once and never repeat yourself.

Why you care: this is the difference between sounding like you and sounding like every other AI-written post on LinkedIn.

Context window

The model's working memory: how much it can hold in mind at once during a conversation. Big, but not infinite. When a chat gets very long, the earliest parts fall out of view.

Why you care: long rambling chats drift. Start fresh conversations for fresh jobs, and keep the permanent stuff in files the AI re-reads each time.

Token

The unit models read and bill in. Roughly three quarters of a word. A thousand tokens is about 750 words.

Why you care: API pricing is per token. It's the AI equivalent of being billed per minute of footage.

Hallucination

When a model states something false with complete confidence: an invented statistic, a made-up quote, a client name that doesn't exist.

Why you care: never send AI-drafted numbers, names or claims to a client without checking them. The model isn't lying, it's pattern-matching, but the damage is the same.

Agent

An AI that doesn't just answer, it acts: reads files, runs steps, checks its own work, carries a task through to a finish. Claude Code working through your folder is an agent. The Prospector Engine is agents in a row.

Why you care: chat is you doing the work with help. Agents are the work happening while you do something else.

MCP

Model Context Protocol. A standard plug that lets an AI connect to your other tools: your CRM, your calendar, your email. One connector per tool, and the AI can read from and act in it.

Why you care: this is how the OS stops being a writing tool and starts touching your actual pipeline.

API

Application programming interface: the way two pieces of software talk to each other directly, without a human clicking buttons. When a tool "integrates with" another tool, it's using an API.

Why you care: the chat subscription and the API are different doors into the same model. Tools you build, like the Prospector Engine, go through the API door.

API key

The password that lets a tool use an API on your account, and bill your card for it.

Why you care: treat keys like bank details. Never paste one into a shared doc, a chat, or a website you don't trust. Anyone holding your key is spending your money as you.

Training cutoff

The date the model's built-in knowledge stops. Anything after it, the model only knows if it searches the web or you tell it.

Why you care: asking a model about last month's news without search turned on is asking it to guess.

The builder's words

Terminal

A window where you type instructions to your computer as text instead of clicking icons. Full explanation two sections down.

Why you care: it's where Claude Code lives, and it's far less frightening than it looks.

Claude Code

Anthropic's AI that runs in the terminal and works directly on the files on your machine. You talk to it in plain English. It reads your folder, writes documents, edits files, and runs the workflows of your OS.

Why you care: this is the operator of your AI OS. Most of what we build over the two days runs through it.

Markdown

Plain text with simple symbols for formatting: # for a heading, ** for bold, - for a bullet. Files end in .md. Readable by any computer, any app, any AI, forever.

Why you care: your whole OS is written in it. No proprietary format, no app that can shut down and take your files with it.

Git

A program on your computer that tracks changes to a folder over time. Every time you save a snapshot, git remembers exactly what changed, when, and lets you wind back to any earlier version.

Why you care: it's version history for your business brain. Break something on Tuesday, restore Monday's version in one line.

Repository (repo)

A folder that git is tracking. Your AI OS folder is one repo.

Why you care: when someone says "the repo", they mean "the folder, with its full history attached".

GitHub

A website that holds a copy of your repo in the cloud. Full explanation below.

Why you care: it's the off-site backup of the most valuable folder you own.

Commit

A saved snapshot of your folder with a short note attached: "added new pricing", "rewrote voice rules". A commit lives on your machine.

Why you care: commits are the save points. Cheap to make, priceless to have.

Push and pull

Push sends your latest commits up to GitHub. Pull brings the latest copy down from GitHub. Full explanation below.

Why you care: push at the end of a working session and your work survives a stolen laptop, a dead drive, a spilled coffee.

Clone

Downloading a full copy of a repo, history and all, onto a machine. New laptop, one clone, and your whole OS is back.

Why you care: your setup stops being trapped in one computer.

Python

The programming language most AI tooling is written in. You won't write it at the workshop, but you'll see its name in error messages and setup guides.

Why you care: only so the word doesn't rattle you. "It's a Python script" means "it's a small program".

Streamlit

A tool that turns a script into a simple web page with buttons, forms and results. Full explanation below.

Why you care: it's how an internal tool becomes something your editor or producer can use without ever seeing a terminal.

Local vs cloud

Local means on your machine: your files, your folder, the terminal. Cloud means on someone else's server: GitHub, your CRM, the model itself.

Why you care: your OS files are local, which means they're private and yours. The backup lives in the cloud. You want both, deliberately.

Deploy

Putting a tool you've built onto a server so it runs all the time at a web address, instead of only when your laptop is open. The Prospector Engine is deployed.

Why you care: "it works on my laptop" becomes "my team can use it from anywhere".

What the terminal actually is

The terminal is the oldest interface on your computer: a plain window where you type an instruction, press enter, and the machine does it. It predates the mouse, the desktop and every icon you've ever clicked. Everything you do by clicking is a polite wrapper around what the terminal does directly.

It looks intimidating because films use it as shorthand for hacking. In practice it's a text conversation with your computer. `cd Documents` means "go to the Documents folder". `ls` means "show me what's in here". That's the level of it.

Here's the part that matters for you: **you no longer need to learn the commands.** Claude Code sits inside the terminal and you talk to it in plain English. "Open my client folder and draft the follow-up email" gets typed by you, and the machinery underneath gets handled. The terminal is the room where the work happens. Claude Code is the colleague in the room. You just have to be willing to walk in.

Think of it like manual mode on a camera. Auto mode (apps, buttons, menus) decides things for you and hides the controls. Manual mode shows you everything and does exactly what you say. You learned manual because the control was worth it. Same trade here, except this time you've got an operator standing next to you who already knows the settings.

What GitHub does, and why push and pull matter

You already run this discipline with footage. No serious production company keeps the only copy of a shoot on one drive. You back up to a second drive, then off-site or to the cloud, because the footage is the asset and drives fail.

Your AI OS deserves the same respect. The folder holding your voice, your pricing, your client records and your workflows will quietly become one of the most valuable folders in your business. Git and GitHub are the backup discipline for it, with one upgrade footage backups never had: **history**.

The working rhythm is three moves:

- **Commit:** save a named snapshot on your machine. "Updated pricing for 2026." Do this whenever you've changed something worth keeping.

- **Push:** send your snapshots up to GitHub, the copy in the cloud. Do this at the end of a working session. From that moment, losing your laptop loses nothing.

- **Pull:** bring the latest copy down from GitHub before you start work, so you're never working on a stale version. It matters most the day you have two machines, or a team member, touching the same folder.

And because every snapshot is kept, you get a time machine for free. The AI rewrote your voice file and made it worse? Wind back to yesterday's version. You want to see how your positioning read six months ago? It's all there, with dates and notes. That's something no Dropbox folder gives you.

One honest note: GitHub was built for programmers and the website shows it. You'll only ever use a sliver of it, and Claude Code will do the commits and pushes for you when you ask. Your job is just to know the rhythm: commit when something's worth keeping, push at the end of the day, pull before you start.

What Streamlit does

Somewhere in the months after the workshop, you'll build a small tool that works. Maybe it scores inbound leads against your ideal client profile. It runs on your laptop, in your terminal, and it's great, and nobody else in your business can use it, because nobody else is going to open a terminal.

Streamlit fixes that. It takes a script and gives it a face: a simple web page with a text box, a button and a results panel. Your producer pastes in the lead's details, clicks the button, gets the score. They never know there's a script behind it, and they don't need to.

That's the whole idea. Terminal tools are for you. Streamlit, and tools like it, are how those tools become usable by your team. It's the difference between owning a camera and handing someone a finished edit.

The two tools you'll use on the day

Everything above is vocabulary. These are the two working tools we put in your hands at the workshop, and between them they use almost every word in this guide. Both are private, members-only tools: you'll be given your login on the day, so don't worry if the links show you a sign-in page before then.

TOOL ONE · PROSPECTOR.DENLENNIE.COM

The Prospector Engine

An AI prospecting tool built specifically for video production companies. You tell it about your business, and it does the afternoon of grind you've been putting off: finds companies that match your ideal client, identifies the decision-makers, pulls their contact details, scores every prospect from 0 to 10, and hands back a ranked list with a personal reason to reach out to each one.

Four steps on the day:

- **Profile.** Paste your website address. The engine drafts your ideal client profile, you correct it. Upload a list of past clients and it learns the pattern from real examples.
- **Find.** It searches for matching companies in your geography. Fifty prospects in three to eight minutes.
- **Enrich and score.** It reads each company's website, finds the decision-makers, and scores the fit. A-tier means start here. D-tier means park it.
- **Export.** A CSV with everything you need for outreach, including a personalised angle per lead referencing real evidence from their site.

One practical note: workshop accounts are time-boxed. Run your real searches during the window and export your CSV. The results are yours to keep.

Words from this guide you'll meet inside it: it's a Streamlit app, deployed on a server, talking to models through APIs, spending tokens per lead, with the small fast model tier doing the scoring. You now know what every one of those means.

TOOL TWO · [NURTURE.DENLENNIE.COM/ENGINE](https://nurture.denlennie.com/engine)

The Nurture Engine

The companion to the Prospector. Prospecting gets you into conversations. This tool keeps you good in them. It drafts replies to your LinkedIn messages in your voice, following the nurture discipline we teach on day two: diagnose before you direct, never sound desperate, end on a question that makes the prospect think rather than a push for a call.

The point isn't speed for its own sake. It's consistency. The fourth follow-up you'd normally fumble, or skip entirely, gets the same calm quality as the first reply you wrote when you were fresh. Leads stop dying of silence.

Words from this guide you'll meet inside it: a system prompt carrying your voice rules, a model drafting against your context, and you verifying before anything leaves the building. The AI drafts. You sign.

THE OPERATING PRINCIPLE

Less AI. Used better.

Every term in this guide describes machinery, and machinery is not the point. Two tools used well beat ten used badly. The owners who win with AI over the next few years won't be the ones who know the most jargon. They'll be the ones whose context is the sharpest: who they serve, how they sound, what they charge, and why. The files carry that. The tools just read them.

Setting up your AI OS: eight things to know before you start

- **Plain text wins.** Keep the OS in markdown files, not in a notes app, not in someone's platform. Apps shut down, get acquired, change pricing. A folder of text files works in 2026 and will work in 2036.
- **One folder, one source of truth.** Everything lives in one place: voice, offers, clients, workflows. The moment your context is scattered across six tools, the AI is guessing again.
- **Back it up before you build on it.** Git plus GitHub from day one. It takes ten minutes to set up and you'll ask Claude Code to do it anyway. Building months of business context with no backup is keeping your only rushes on one drive.
- **API keys are bank details.** Never in a shared doc, never in an email, never pasted into a tool you don't trust. If a key leaks, you revoke it and issue a new one, the same day.
- **Know where client data goes.** Before you paste a client's contract or financials into any AI tool, know what that tool does with it. Use business-tier accounts with training turned off. Your clients' confidentiality is your reputation.
- **Start with one workflow that hurts weekly.** Proposals, follow-ups, call summaries: pick the one that costs you the most hours, get it working end to end, then add the next. Ten half-built workflows is the tool circus you came here to escape.
- **Correct the AI in writing, once.** When it gets your voice or your facts wrong, don't just fix the draft. Fix the file it read from. That's the entire trick of an OS that improves: corrections compound, chat messages evaporate.
- **Verify before it leaves the building.** Anything with numbers, names, dates or claims gets a human read before a client sees it. The AI drafts. You sign.

